Shouri Jalandar (CS23090)

MI Lab-7

Aim:- To apply support Vector Machine for classification analyze the impact of different Kernels on model accuracy

```
import pandas as pd
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

```
path= "/content/drive/MyDrive/ML_Dataset/pulsar_stars.csv"
df=pd.read_csv(path)
df.head(5)
```

	Mean of the integrated profile	Standard deviation of the integrated profile	Excess kurtosis of the integrated profile	Skewness of the integrated profile	Mean of the DM-SNR curve	Standard deviation of the DM-SNR curve	Excess kurtosis of the DM-SNR curve	Skewness of the DM-SNR curve	target_class
0	140.562500	55.683782	-0.234571	-0.699648	3.199833	19.110426	7.975532	74.242225	0
1	102.507812	58.882430	0.465318	-0.515088	1.677258	14.860146	10.576487	127.393580	0
2	103.015625	39.341649	0.323328	1.051164	3.121237	21.744669	7.735822	63.171909	0
3	136.750000	57.178449	-0.068415	-0.636238	3.642977	20.959280	6.896499	53.593661	0

```
df.shape
```

```
(17898, 9)
```

```
df.describe()
```

	Mean of the integrated profile	Standard deviation of the integrated profile	Excess kurtosis of the integrated profile	Skewness of the integrated profile	Mean of the DM-SNR curve	Standard deviation of the DM-SNR curve	Excess kurtosis of the DM-SNR curve	Skewness of the DM-SNR curve	target_class
count	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000
mean	111.079968	46.549532	0.477857	1.770279	12.614400	26.326515	8.303556	104.857709	0.091574
std	25.652935	6.843189	1.064040	6.167913	29.472897	19.470572	4.506092	106.514540	0.288432
min	5.812500	24.772042	-1.876011	-1.791886	0.213211	7.370432	-3.139270	-1.976976	0.000000
25%	100.929688	42.376018	0.027098	-0.188572	1.923077	14.437332	5.781506	34.960504	0.000000
50%	115.078125	46.947479	0.223240	0.198710	2.801839	18.461316	8.433515	83.064556	0.000000

```
# Remove leading and trailing spaces from column names
df.columns = df.columns.str.strip()
print(df.columns)
```

```
Index(['Mean of the integrated profile',
       'Standard deviation of the integrated profile',
       'Excess kurtosis of the integrated profile',
       'Skewness of the integrated profile', 'Mean of the DM-SNR curve',
       'Standard deviation of the DM-SNR curve',
       'Excess kurtosis of the DM-SNR curve', 'Skewness of the DM-SNR curve',
       'target_class'],
```

```
      dtype='object')
```

```
df = df.rename(columns={
    'Mean of the integrated profile': 'MIP',
    'Standard deviation of the integrated profile': 'SDIP',
    'Excess kurtosis of the integrated profile': 'EKIP',
    'Skewness of the integrated profile': 'SKIP',
    'Mean of the DM-SNR curve': 'MDSNR',
    'Standard deviation of the DM-SNR curve': 'SDSNR',
    'Excess kurtosis of the DM-SNR curve': 'EKSNR',
    'Skewness of the DM-SNR curve': 'SKSNR',
    'target_class': 'Target'
})

print(df.columns)
```

```
Index(['MIP', 'SDIP', 'EKIP', 'SKIP', 'MDSNR', 'SDSNR', 'EKSNR', 'SKSNR',
       'Target'],
      dtype='object')
```

```
df.describe()
```

	MIP	SDIP	EKIP	SKIP	MDSNR	SDSNR	EKSNR	SKSNR	Target
count	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000
mean	111.079968	46.549532	0.477857	1.770279	12.614400	26.326515	8.303556	104.857709	0.091574
std	25.652935	6.843189	1.064040	6.167913	29.472897	19.470572	4.506092	106.514540	0.288432
min	5.812500	24.772042	-1.876011	-1.791886	0.213211	7.370432	-3.139270	-1.976976	0.000000
25%	100.929688	42.376018	0.027098	-0.188572	1.923077	14.437332	5.781506	34.960504	0.000000
50%	115.078125	46.947479	0.223240	0.198710	2.801839	18.461316	8.433515	83.064556	0.000000
75%	127.085938	51.023202	0.473325	0.927783	5.464256	28.428104	10.702959	139.309330	0.000000
max	192.617188	98.778911	8.069522	68.101622	223.392141	110.642211	34.539844	1191.000837	1.000000

```
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
df.columns
```

```
Index(['MIP', 'SDIP', 'EKIP', 'SKIP', 'MDSNR', 'SDSNR', 'EKSNR', 'SKSNR',
       'Target'],
      dtype='object')
```

```
print(df['Target'].value_counts())
```

```
Target
0    16259
1     1639
Name: count, dtype: int64
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
X_train
```

Show hidden output

```
X_test
```

Show hidden output

```
X
```

	MIP	SDIP	EKIP	SKIP	MDSNR	SDSNR	EKSNR	SKSNR
0	140.562500	55.683782	-0.234571	-0.699648	3.199833	19.110426	7.975532	74.242225
1	102.507812	58.882430	0.465318	-0.515088	1.677258	14.860146	10.576487	127.393580
2	103.015625	39.341649	0.323328	1.051164	3.121237	21.744669	7.735822	63.171909
3	136.750000	57.178449	-0.068415	-0.636238	3.642977	20.959280	6.896499	53.593661
4	88.726562	40.672225	0.600866	1.123492	1.178930	11.468720	14.269573	252.567306
...	...	...	...	...	...	...	...	...
17893	136.429688	59.847421	-0.187846	-0.738123	1.296823	12.166062	15.450260	285.931022
17894	122.554688	49.485605	0.127978	0.323061	16.409699	44.626893	2.945244	8.297092
17895	119.335938	59.935939	0.159363	-0.743025	21.430602	58.872000	2.499517	4.595173
17896	114.507812	53.902400	0.201161	-0.024789	1.946488	13.381731	10.007967	134.238910
17897	57.062500	85.797340	1.406391	0.089520	188.306020	64.712562	-1.597527	1.429475

17898 rows × 8 columns

Vector Machine

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
svc=SVC()
svc.fit(X_train,y_train)
y_pred=svc.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
print('Model accuracy score with default hyperparameters: {0:0.4f}'. format(accuracy))
```

```
Model accuracy score with default hyperparameters: 0.9790
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

```
print('Confusion Matrix')
cm = confusion_matrix(y_pred,y_test)
print(confusion_matrix(y_pred,y_test))
```

```
Confusion Matrix
[[4863   92]
 [  21  394]]
```

```
print('Accuracy Score')
print(accuracy_score(y_pred,y_test))
```

```
Accuracy Score
0.9789571694599628
```

```
print('Classification Report')
print(classification_report(y_pred,y_test))
```

```
Classification Report
              precision    recall  f1-score   support

           0       1.00      0.98      0.99      4955
           1       0.81      0.95      0.87       415

    accuracy                           0.98      5370
   macro avg       0.90      0.97      0.93      5370
weighted avg       0.98      0.98      0.98      5370
```

```
svc=SVC(C=1.0)
svc.fit(X_train,y_train)
y_pred=svc.predict(X_test)
print('Model accuracy score with rbf kernel and c=1.0 : {0:.4f}'.format(accuracy))
```

```
Model accuracy score with rbf kernel and c=1.0 : 0.9790
```

```
svc=SVC(C=100.0)
svc.fit(X_train,y_train)
y_pred=svc.predict(X_test)
print('Model accuracy score with rbf kernel and c=100.0 : {0:.4f}'.format(accuracy))
```

```
Model accuracy score with rbf kernel and c=100.0 : 0.9790
```

```
svc=SVC(C=1000.0)
svc.fit(X_train,y_train)
y_pred=svc.predict(X_test)
print('Model accuracy score with rbf kernel and c=100.0 : {0:.4f}'.format(accuracy))
```

```
Model accuracy score with rbf kernel and c=100.0 : 0.9790
```

SVM with linear

```
linear_svc=SVC(kernel='linear',C=1.0)
linear_svc.fit(X_train,y_train)
y_pred_test=linear_svc.predict(X_test)
linear_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with linear kernel and c=1.0 : {0:.4f}'.format(linear_accuracy))
```

```
Model accuracy score with linear kernel and c=1.0 : 0.9788
```

```
linear_svc=SVC(kernel='linear',C=100.0)
linear_svc.fit(X_train,y_train)
y_pred_test=linear_svc.predict(X_test)
linear_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with linear kernel and c=100.0 : {0:.4f}'.format(linear_accuracy))
```

```
Model accuracy score with linear kernel and c=1.0 : 0.9786
```

```
linear_svc=SVC(kernel='linear',C=1000.0)
linear_svc.fit(X_train,y_train)
y_pred_test=linear_svc.predict(X_test)
linear_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with linear kernel and c=1000.0 : {0:.4f}'.format(linear_accuracy))
```

```
Model accuracy score with linear kernel and c=1000.0 : 0.9786
```

SVM with polynomial

```
poly_svc=SVC(kernel='poly',C=1.0)
poly_svc.fit(X_train,y_train)
y_pred_test=poly_svc.predict(X_test)
poly_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with polynomial kernel and c=1.0 : {0:.4f}'.format(poly_accuracy))
```

```
Model accuracy score with polynomial kernel and c=1.0 : 0.9769
```

```
poly_svc=SVC(kernel='poly',C=100.0)
poly_svc.fit(X_train,y_train)
y_pred_test=poly_svc.predict(X_test)
poly_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with polynomial kernel and c=100.0 : {0:.4f}'.format(poly_accuracy))
```

```
Model accuracy score with polynomial kernel and c=100.0 : 0.9793
```

```
poly_svc=SVC(kernel='poly',C=1000.0)
poly_svc.fit(X_train,y_train)
y_pred_test=poly_svc.predict(X_test)
poly_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with polynomial kernel and c=1000.0 : {0:.4f}'.format(poly_accuracy))
```

```
Model accuracy score with polynomial kernel and c=1000.0 : 0.9793
```

svm with sigmoid

```
sigmoid_svc=SVC(kernel='sigmoid',C=1.0)
sigmoid_svc.fit(X_train,y_train)
y_pred_test=sigmoid_svc.predict(X_test)
```

```
sigmoid_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with sigmoid kernel and c=1.0 : {0:.4f}'.format(sigmoid_accuracy))
```

```
Model accuracy score with sigmoid kernel and c=1.0 : 0.8773
```

```
sigmoid_svc=SVC(kernel='sigmoid',C=100.0)
sigmoid_svc.fit(X_train,y_train)
y_pred_test=sigmoid_svc.predict(X_test)
sigmoid_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with sigmoid kernel and c=100.0 : {0:.4f}'.format(sigmoid_accuracy))
```

```
Model accuracy score with sigmoid kernel and c=100.0 : 0.8773
```

```
sigmoid_svc=SVC(kernel='sigmoid',C=1000.0)
sigmoid_svc.fit(X_train,y_train)
y_pred_test=sigmoid_svc.predict(X_test)
sigmoid_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with sigmoid kernel and c=1000.0 : {0:.4f}'.format(sigmoid_accuracy))
```

```
Model accuracy score with sigmoid kernel and c=1000.0 : 0.8773
```

```
print('Confusion Matrix')
cm = confusion_matrix(y_pred,y_test)
print(confusion_matrix(y_pred,y_test))
```

```
Confusion Matrix
[[4863   92]
 [  21  394]]
```

Start coding or generate with AI.